

Log Anomaly Detection

A self-monitoring delivery pipeline, explained from zero.

Everything in this project, what each piece is for, and why it was built the way it was.

Section	What it covers
Part 1	What you built, in plain language
Part 2	The detection problem, and the machine learning behind it
Part 3	Every concept used, from zero: containers, CI/CD, cloud, IaC
Part 4	The five stages, walked through one at a time
Part 5	Every file in the repository and what it does
Part 6	The bugs we hit, and what each one teaches
Part 7	Running it, demonstrating it, and tearing it down
Part 8	Questions you should be able to answer about it
Part 9	Glossary

Part 1 · What you built

Read this part first. Everything else is detail.

The one-sentence version

You have a service that reads log files and points out the parts that look wrong. Around it, you built the machinery that tests it, packages it, provisions cloud servers for it, deploys it automatically, and watches it in production. Then you pointed the service at the logs that machinery produces, so it monitors its own deployment.

Why that last sentence is the whole point

Plenty of people can say "I containerised an application". The thing that makes this project memorable is the loop: the pipeline that deploys the detector produces logs, and the detector reads those logs. If a deployment misbehaves, the tool catches it.

When you describe this project, lead with that.

The two halves

It helps to hold these separately in your head, because they are genuinely different kinds of work.

The application	The delivery pipeline
Python. Reads logs, groups similar lines together, scores each group for how unusual it is, and explains the suspicious ones.	Everything else. Testing, packaging, cloud servers, automated deployment, monitoring, alerting.
Existed before this project started. Roughly 1,000 lines.	Built during this project. Roughly 2,000 lines across YAML, HCL and shell.
Interesting because of what it measures.	Interesting because of what happens when it fails.

For an infrastructure or DevOps role, the right-hand column is what you are being hired for. The left-hand column is the thing that gives the pipeline something real to carry.

What actually runs, end to end

Here is the full journey of one code change, from your keyboard to a running server. Every step of this is automated and every step actually happened.

1. You edit a file and run `git push`.
2. **GitHub Actions** wakes up and runs the tests on two different Python versions, checks code formatting, checks the shell scripts, checks the infrastructure code, and builds the application into a container image.
3. It starts that container and asks it whether it is healthy. If the container builds but does not run, the build fails here.
4. The image is pushed to a **registry** (a warehouse for container images), tagged with the exact commit it came from.
5. The pipeline proves its identity to Amazon Web Services using a short-lived token, opens the server's SSH port for its own address only, and connects.

6. It copies the deployment script over and runs it. That script pulls the new image, stops the old container, starts the new one, and waits up to 30 seconds for the service to report itself healthy.
7. It checks that the running service reports the **exact commit** it was supposed to deploy. If it is healthy but running the wrong version, that counts as a failure.
8. If anything failed, the previous image is put back and the service keeps serving. The pipeline reports whether the rollback worked, because "the change did not ship" and "nobody is serving traffic" need different responses.
9. The SSH port is closed again, whatever happened.
10. Every 15 minutes after that, a timer on the server feeds the service's own logs back through the detector and publishes how many suspicious windows it found. An alarm watches that number.

What it costs

Nothing, if you follow the rules. Every AWS resource was chosen to sit inside the free tier, and there are three guardrails: a budget alert that fires at one cent, automatic deletion of stored logs after 30 days, and an expiry on the log group. The single most important habit is running `terraform destroy` when you are not actively working on it.

The two things that will cost you money

A **NAT gateway** is about \$32 a month. This project deliberately avoids one by putting the server in a public subnet with a tight firewall.

An **Elastic IP** (a fixed public address) is free while attached to a running server, and billed by the hour the moment that server stops. If you stop the instance rather than destroying it, you start paying.

Part 2 - The detection problem

What the application actually does, and the machine learning behind it.

Why log anomaly detection is hard

A busy system produces millions of log lines a day. Almost all of them are routine. Somewhere in there are the few that matter. Nobody can read them all, and simple keyword searching for words like "error" misses most real problems while drowning you in harmless ones.

The deeper difficulty is that **a single line is usually not enough to judge**. Take this line:

```
kernel: page allocation failure order:2
```

On its own, that is ordinary. Linux systems emit it routinely. What makes it worth investigating is context: what came before it, and how many times it fired in a short span. One is noise. Forty in ten seconds is an incident.

Step one: turn lines into templates

Raw log lines are nearly all unique, because they contain variable parts - timestamps, process IDs, addresses. These two lines are the same event:

```
core.1234 written  
core.5678 written
```

A tool called **Drain3** spots that and collapses both into one **template**:

```
core.<*> written
```

This is the single most important preprocessing step. It turns millions of near-identical strings into a few hundred distinct event types. In this project, 2,000 log lines collapse to 105 templates.

How Drain3 decides

It builds a tree keyed on the first few tokens of each line and the line's length, then compares candidates by what fraction of their tokens match. If the match is above a **similarity threshold**, the line joins that cluster and any differing tokens become wildcards. If not, it starts a new cluster.

That threshold matters, and it caused a real bug in this project. More on that in Part 6.

Step two: group lines into windows

Because a single line rarely tells you enough, lines are batched into **windows** - consecutive runs of, say, 50 lines. Each window becomes one row of data, described by a **count vector**: one number per template, saying how many times that template appeared in the window.

Window	template A	template B	template C
1	48	2	0
2	47	3	0
3	12	1	37

Window 3 looks nothing like the others - not because any single line is alarming, but because the mixture changed.

These vectors are normalised so that a long window does not look unusual purely for being long.

This step is where nearly all the accuracy comes from

Measured on the benchmark data in this repository, scoring individual lines gives an F1 score of 0.133. Scoring windows of 50 lines gives **0.765** - a 5.8x improvement.

Everything else in the pipeline is a rounding error next to that. It is worth being able to say so, and to explain why: line-level scoring throws away context by construction, so no amount of clever embedding rescues it.

Step three: score each window

Now it is an ordinary outlier-detection problem: given a pile of numeric rows, which ones do not look like the rest? Three methods are implemented, and comparing them is deliberate.

Method	Idea	Result on the benchmark
Isolation Forest	Randomly split the data. Points that get isolated in few splits are odd.	AUC 0.824, F1 0.765 - best overall
k-nearest-neighbour distance	Normal events recur, so they sit in crowded regions. Rare ones sit far from everything.	AUC 0.752, F1 0.588
Template rarity	No vectors at all. Just score by how infrequent the template is.	The control. Beat the fancy methods at line level.

That third one exists specifically so the claim "the sophisticated approach helped" is testable rather than assumed. On this data, at line level, it was not true - the simple frequency heuristic matched or beat the embedding pipeline. Reporting that honestly is worth more than hiding it.

Unsupervised, and why that matters

The benchmark data has labels saying which lines were real faults. Those labels are used **only to measure accuracy afterwards**, never to train. In production there are no labels - nobody has told you in advance which of today's logs are the bad ones. If you trained on labels you would have built a different, much easier thing: a classifier. Being clear about that distinction is a mark of someone who understands the problem.

Step four: explain the result

A detector that says "window 47 scores 0.91" is useless to the engineer who has been paged at 3am. The last stage sends the flagged window's templates to a language model, along with templates from a **normal** window for contrast, and asks for a plain-language summary, a severity, and a suggested next step.

The language model does not do the detecting

This is a deliberate architectural decision and a good one to be able to defend. Detection is statistical, deterministic and reproducible - run it twice, get the same answer. The model only **explains** something already flagged.

Putting a language model in the detection path would make results non-reproducible, impossible to audit, and expensive per log line.

Two supporting details worth knowing. The prompt always includes a normal window for **contrast** - without it the model just paraphrases the logs back at you instead of reasoning about them. And responses are **cached by content hash**, because templates repeat constantly and you should never pay twice for the

same explanation.

A stub implementation ships in the code, so the entire test suite runs with no API key and no network. A test suite that needs a credential is a test suite that quietly stops being run.

Part 3 - Every concept, from zero

If a term used in this project is unfamiliar, it is defined here.

3.1 Containers, and the problem they solve

Your application needs Python 3.12, a specific version of scikit-learn, and a handful of other libraries. The server does not have them. Your laptop has different versions. A colleague has different ones again. Historically this produced the oldest excuse in software: it works on my machine.

A **container** solves it by packaging the application together with its entire environment - the Python interpreter, every library, the data files it needs - into a single image that runs identically anywhere Docker is installed.

Image versus container

Term	What it is	Analogy
Image	A read-only package. Built once, stored, copied around.	A recipe, or a class in code
Container	A running instance of an image. You can start several from one image.	The cooked dish, or an object

The Dockerfile

A Dockerfile is the build recipe. This project's, annotated:

```
FROM python:3.12-slim # start from a minimal Python 3.12 system

WORKDIR /app # work inside /app from here on

COPY requirements.txt . # copy just the dependency list...
RUN pip install -r requirements.txt # ...and install it

COPY src/ ./src/ # then copy the actual source code
COPY data/ ./data/

USER appuser # stop running as root
EXPOSE 8000 # this container serves on port 8000
HEALTHCHECK ... # how Docker asks whether it is alive
CMD ["uvicorn", "loganomaly.api:app", "--host", "0.0.0.0"]
```

Why the dependency list is copied before the source code

Docker caches each step. If a step's inputs have not changed, it reuses the previous result instead of redoing the work.

Installing scikit-learn and scipy takes about a minute. Your source code changes constantly; your dependency list barely ever does. Copying requirements.txt first and installing before copying the code means an ordinary code change reuses the cached install and the build takes seconds. Swapping those two lines would reinstall everything on every commit.

Running as a non-root user

By default a container runs as root. If someone finds a way to execute code inside it, root gives them far more room to move. The Dockerfile creates an unprivileged user and switches to it. Nothing here needs root at runtime, so nothing is lost by dropping it.

3.2 Registries

Once an image is built, it has to reach the server. A **registry** is a warehouse for images: you push from the build machine and pull on the server. Docker Hub is the best-known one. This project uses **GitHub Container Registry** (ghcr.io), which comes free with the repository.

Tags, and why there are two

```
ghcr.io/gibmedacookie09/log-anomaly-detection:v0.1.0 <- for humans
ghcr.io/gibmedacookie09/log-anomaly-detection:sha-05424d1 <- for machines
```

Version tags are what a person asks for, but they can be moved or deleted - latest is famously whatever someone pushed most recently. The **commit SHA tag** is immutable. Six months from now, a container running sha-05424d1 traces to exactly one commit with certainty. That is why the deployment records the SHA tag rather than the version tag.

3.3 Continuous Integration and Continuous Deployment

	What it means	In this project
CI	Every change is tested automatically the moment it is pushed, so problems surface in minutes rather than at release time.	ci.yml - seven jobs on every push and pull request
CD	Changes that pass CI are released automatically, with no manual steps.	deploy.yml - runs on every merge to main

GitHub Actions vocabulary

- **Workflow** - a YAML file in .github/workflows/. One file, one pipeline.
- **Trigger** (on:) - what starts it: a push, a pull request, a tag, a manual click, or another workflow calling it.
- **Job** - a unit that gets its own fresh virtual machine. Jobs run in parallel unless you declare dependencies with needs: .
- **Step** - one command or one reusable action inside a job. Steps run in order on the same machine.
- **Runner** - the throwaway virtual machine a job runs on. You get a clean one every time, which is why nothing persists between runs unless you cache it.
- **Matrix** - runs the same job several times with different values. This project tests Python 3.11 and 3.12 simultaneously.

What a workflow file looks like

```
name: CI

on: # WHEN to run
push:
branches: ["**"]
pull_request:
```



```

jobs:
  test: # a job
  runs-on: ubuntu-latest # on a fresh Ubuntu machine
  strategy:
  matrix:
  python-version: ["3.11", "3.12"] # run this job twice
  steps:
  - uses: actions/checkout@v4 # fetch the code
  - uses: actions/setup-python@v5 # install Python
  with:
  python-version: (matrix value)
  cache: pip # restore the download cache
  - run: pip install -r requirements-dev.txt
  - run: pytest

```

Caching, and why it earns its place

Every job starts on a clean machine, which means downloading every dependency every time. scikit-learn, scipy and pandas are large.

cache: pip saves the download folder and restores it next run, keyed on a hash of the requirements files. When those files have not changed the cache is reused; when they change, the key changes and it downloads fresh. Correctness is preserved automatically - you cannot accidentally use a stale cache.

3.4 The cloud, and the five services used here

The cloud means renting computers and services from someone else's data centre, by the hour, through an API. This project uses Amazon Web Services.

Service	What it is	Used here for
EC2	Virtual machines. Pick a size and an operating system, get a server.	One t3.micro running the container
S3	Object storage. Buckets of files addressed by key. Not a filesystem - you put and get whole objects.	Ingested logs and detector results
IAM	Identity and Access Management. Who may do what.	A role granting the server access to exactly one bucket and one log group
CloudWatch	Logs, metrics and alarms.	Container output, request metrics, four alarms
SNS	Notifications. Publishers send to a topic, subscribers receive.	Delivering alarm emails

Instance types and the free tier

t3.micro means two virtual CPUs and 1 GB of memory - small, and free for 750 hours a month, which covers one instance running continuously. Anything larger bills immediately.

Check which free tier your account is on

AWS changed the model. Older accounts get twelve months of monthly allowances. Newer accounts get a fixed credit grant that expires instead. Which applies depends on when the account was opened.

The sizing here is conservative under either, but do not assume the twelve-month version without checking the Billing console.

Security groups

A **security group** is a firewall attached to the server. It is default-deny: nothing gets in unless a rule allows it. This project opens exactly two ports, both only to your own address:

```
port 22 (SSH) from 103.120.251.182/32
port 8000 (API) from 103.120.251.182/32
all outbound allowed - needed to pull images and ship logs
```

The /32 is CIDR notation for exactly one address. /24 would mean 256 addresses. 0.0.0.0/0 means the entire internet.

3.5 Infrastructure as Code, and Terraform

You could create that server by clicking through the AWS console, and many people do. The problems arrive later: nobody remembers what was clicked, it cannot be code-reviewed, it cannot be recreated reliably, and there is no record of why any setting is the way it is.

Infrastructure as Code means describing infrastructure in files kept in version control. **Terraform** reads those files and makes reality match.

How Terraform thinks

You describe **what you want**, not the steps to get there. Terraform compares your description against a **state file** recording what it built last time, computes the difference, and applies it.

Command	What it does
terraform init	Downloads the provider plugins. Once per project.
terraform plan	Shows exactly what it would change, and changes nothing. Always read this before applying.
terraform apply	Makes the changes.
terraform destroy	Deletes everything it created.

The language

Terraform files use HCL. Four things appear in them:

```
variable "instance_type" { # an input you can set
  type = string
  default = "t3.micro"
}

resource "aws_instance" "app" { # something to CREATE
  instance_type = var.instance_type
  ami = data.aws_ami.al2023.id
```

```

}

data "aws_ami" "al2023" { # something to LOOK UP, not create
  most_recent = true
  owners = ["amazon"]
}

output "public_ip" { # something to print when finished
  value = aws_instance.app.public_ip
}

```

Resources reference each other by name. `aws_instance.app.public_ip` means "the public IP of the instance called `app`". Terraform reads those references to work out what must exist before what - you never specify an order.

The state file matters, and is slightly dangerous

`terraform.tfstate` records every resource Terraform manages. Lose it and Terraform no longer knows what it built; it will try to create everything again alongside what is already there.

It also holds resource IDs and can hold secrets, so it is gitignored here. For a team you would keep it in a shared S3 bucket instead - `versions.tf` carries a commented-out block for exactly that, along with a note on why it is not enabled: the bucket would have to exist before the configuration that creates buckets can run.

3.6 SSH and key pairs

SSH gives you a command line on a remote machine. It authenticates with a **key pair** rather than a password:

- The **private key** stays on your machine and is never shared. Anyone holding it can log in as you.
- The **public key** is installed on the server and can be shared freely.

The server challenges you to prove you hold the private key without ever seeing it. This is why the deployment pipeline needs your private key stored as a GitHub secret: it has to authenticate as you in order to deploy.

3.7 Observability: logs, metrics and alarms

	What it is	Good for
Logs	Individual events with full detail. "Request to /health returned 503 in 2ms."	Understanding one specific incident
Metrics	Numbers over time. "47 requests in the last five minutes."	Spotting trends, and triggering alarms
Alarms	A rule on a metric that notifies a human.	Being told, instead of watching

Logs are detailed but expensive to store and slow to query. Metrics are cheap and fast but discard the detail. This project derives metrics **from** its logs, which is a common and efficient pattern: the application does one thing - write a log line - and CloudWatch turns those lines into numbers.

The structured logging trick

The application writes one JSON line per request:

```
{"ts":"2026-08-26T17:57:15Z","event":"request","method":"GET",  
"path":"/health","status":200,"duration_ms":2.48,"sha":"05424d1"}
```

A CloudWatch **metric filter** watches the log group and turns matching lines into numbers. A pattern such as `{ $.status >= 500 }` counts server errors. No agent to install, and no metrics code in the application.

The log format is a contract

Rename a field and the metric filters silently stop matching. They do not error - they simply produce no data, which on a dashboard is indistinguishable from a service receiving no traffic.

That is why `tests/test_api.py` asserts the exact field names. It is the kind of test that looks pointless right up until the day it saves you.

3.8 OIDC, and why it replaced a stored password

The deployment needs permission to change AWS settings. The obvious approach is to create an AWS access key and store it as a GitHub secret. That works, and it is what most tutorials show. It also leaves a long-lived credential sitting somewhere it does not need to be, never expiring unless someone remembers to rotate it.

OIDC (OpenID Connect) replaces the stored password with a proof of identity:

1. GitHub mints a short-lived signed token describing the job - which repository, which branch, which environment.
2. The job presents that token to AWS.
3. AWS verifies the signature against GitHub's published keys, checks the token's claims against a **trust policy** you wrote, and if they match, issues temporary credentials that expire with the job.

No password is stored anywhere. The trust policy is what makes it safe: without a condition pinning the repository, **any** GitHub repository in the world could ask AWS for your credentials. This project's policy pins the audience, the repository, and the environment.

Part 4 · The five stages

What each stage adds, and why it exists.

Stage 1 - Continuous Integration

File: `.github/workflows/ci.yml`. **Runs on:** every push, every pull request. **Seven jobs.**

Job	What it checks
test (3.11)	The full test suite on Python 3.11
test (3.12)	The same suite on 3.12 - <code>fail-fast: false</code> , so both report
lint (ruff)	Code style and formatting
deploy script	shellcheck on every shell script, plus the seven rollback scenarios
terraform	<code>fmt</code> - check and validate against the real AWS provider schema
docker build	Builds the image, starts it , and polls <code>/health</code>
CI passed	Aggregate - green only if all six above are

Three decisions worth defending

The Docker job runs the container, it does not only build it. A successful build proves the image compiles; it says nothing about whether the process comes up. The smoke test hits the same `/health` endpoint that gates deployments, so CI and production agree on what "working" means.

There is one aggregate check, not six. Branch protection points at `CI passed`. Adding a Python version to the matrix therefore does not require editing repository settings - a small thing that stops settings drifting away from the code.

The rollback logic is tested here. It is the least-exercised branch in the whole project: in normal operation it never runs. Without a test, the first time you would discover whether it works is during an outage.

Stage 2 - Image registry

File: `.github/workflows/release.yml`. **Runs on:** pushing a tag starting with `v`.

```
git tag v0.1.0 && git push origin v0.1.0
```

Builds and publishes to `ghcr.io` with both a version tag and an immutable SHA tag. Three details matter:

- **The tag runs CI first.** A tag push does not match `ci.yml`'s branch filter, so without a gate you could tag a broken commit and publish it. `release.yml` calls `ci.yml` as a **reusable workflow** and only publishes if it passes.
- **Provenance is verified, not asserted.** After pushing, the workflow pulls the image back by its SHA tag, starts it, and fails the release unless `/health` reports the commit it was built from.
- **Authentication uses the automatic `GITHUB_TOKEN`** - no personal access token to create, rotate or leak.

Stage 3 - Infrastructure as code

Directory: `infra/`. Twenty-eight AWS resources.

File	What it creates
<code>s3.tf</code>	Bucket with public access blocked, encryption on, ACLs disabled, lifecycle expiry
<code>iam.tf</code>	Instance role and two scoped policies
<code>network.tf</code>	Security group and its rules; looks up the default VPC
<code>ec2.tf</code>	The <code>t3.micro</code> , its AMI lookup, key pair, optional Elastic IP
<code>cloudwatch.tf</code>	Log group with explicit retention
<code>monitoring.tf</code>	Metric filters, four alarms, SNS topic and subscription
<code>budget.tf</code>	The \$1 budget alert
<code>oidc.tf</code>	GitHub OIDC provider and the deploy role
<code>user_data.sh.tftpl</code>	First-boot script: Docker, directories, environment

Least privilege, concretely

This is the part most worth being able to explain, because "least privilege" is a phrase everyone uses and few people demonstrate. The instance role grants three things:

Action	Scoped to
<code>s3:ListBucket</code>	this bucket, conditioned on the <code>logs/</code> and <code>results/</code> prefixes
<code>s3:GetObject</code> , <code>PutObject</code> , <code>DeleteObject</code>	those two prefixes only
<code>logs:CreateLogStream</code> , <code>PutLogEvents</code>	this one log group

What that deliberately excludes is the interesting half:

- **No AWS managed policies.** `AmazonS3FullAccess` would have been one line and would have granted this instance every bucket in the account, including ones that do not exist yet.
- **No resource wildcards.** Every statement names an ARN.
- **No `CreateLogGroup`.** Terraform creates the group, so the instance only ever needs to append to it. An application that can create log groups can create a thousand of them.
- **Prefix-scoped listing.** `s3:ListBucket` is a bucket-level action and cannot be restricted by object ARN, so it carries an `s3:prefix` condition instead. Without that, "scoped to two prefixes" would still allow enumerating the entire bucket.

IMDSv2, and the Capital One breach

Every EC2 instance can reach a metadata service at `169.254.169.254` that hands out its own credentials. In version 1 of that service, a plain HTTP GET is enough. That means any **server-side request forgery** bug in your application - anywhere you fetch a URL a user controls - can be pointed at the metadata endpoint to steal the instance's AWS credentials. This is the mechanism behind the 2019 Capital One breach.

Version 2 requires obtaining a token by PUT first, which an SSRF cannot normally do. This project sets `http_tokens = "required"`, so v1 is refused outright, and sets a hop limit of 1 so a **container** cannot reach it at all.

Stage 4 - Continuous deployment

Files: `.github/workflows/deploy.yml` and `deploy/deploy.sh`. **Runs on:** every merge to main.

The interesting part is not the happy path. It is what happens when the deploy is bad.

```
pull the new image <- BEFORE stopping anything, so a registry
| failure cannot cause an outage
start the new container
|
poll /health for 30s ----- fails --> restore the previous image
| passes |
verify build.sha matches --- wrong --> poll /health for 90s
| matches | passes | fails
record it as last-known-good exit 1 exit 2
exit 0 (service up) (SERVICE DOWN)
```

Why the exit codes differ

Code	Meaning	What you do about it
0	Deployed and healthy	Nothing
1	Deploy failed, previous image restored and serving	Annoying. Look at it tomorrow.
2	Deploy failed and the rollback failed	Nobody is serving traffic. Look at it now.

Collapsing 1 and 2 into a single "failed" throws away the only thing you actually need to know at three in the morning.

Health alone is not sufficient

If the new container had silently failed to replace the old one, `/health` would answer perfectly happily - from the **previous** version - and the deploy would report success having shipped nothing. So `/health` reports `build.sha`, and the deploy fails unless it matches the commit being deployed.

The runner IP problem

SSH is restricted to your home address. GitHub's runners get an unpredictable address from an enormous pool, so they cannot reach port 22 at all. Three obvious fixes are all bad:

Option	Why not
Open SSH to 0.0.0.0/0	Found by scanners within minutes
Allow-list GitHub's published ranges	Thousands of CIDRs that change, against a default limit of 60 rules per group
Store an AWS access key in GitHub	A long-lived credential somewhere it does not need to be

What it does instead: the job proves its identity with a short-lived OIDC token, opens port 22 for **its own runner address only**, deploys, and revokes that rule in an `always( )` step so the port closes even if the deploy fails or the job is cancelled.

Stage 5 - Observability, and the loop

Files: monitoring/, scripts/self_monitor.py, deploy/self-monitor.timer.

```
request -> structured JSON log -> stdout
| docker awslogs driver
v
CloudWatch Logs
| |
metric filters | | self_monitor.py reads
v | it back every 15 min
RequestCount / ServerErrors v
/ LatencyMs fit on the older portion,
| score the recent portion
v |
alarms -> SNS <-----+
|
AnomaliesDetected metric
+ results written to S3
```

The four alarms

Alarm	Fires when
api-5xx-rate	Server error rate above 5% for two periods
instance-cpu	CPU above 80% for two periods
instance-status	EC2's own health check fails
anomalies-detected	The detector flags more than three windows in 15 min

Two periods rather than one on the CPU alarm is deliberate: a t3.micro briefly pegs its CPU on every deploy while the pipeline fits, and paging on that would be pure noise. `treat_missing_data = "notBreaching"` on the rate alarms matters for the same reason - no traffic is not an outage, and an alarm that goes to `INSUFFICIENT_DATA` every quiet night trains you to ignore it.

The self-monitoring run

A systemd timer runs every 15 minutes. It pulls the last two hours from CloudWatch, fits a detector on the older three quarters, scores the most recent quarter against it, writes both the raw batch and the verdict to S3, and publishes the count of flagged windows as a metric that has an alarm on it.

The threshold comes from the baseline's own score distribution - the 95th percentile - rather than a hardcoded number, so it adapts to how noisy the service normally is.

Fitting and scoring must be split chronologically

If you fit the detector on the same lines you then score, it convinces itself everything is normal - the anomaly ends up inside the model of normality. The older portion trains; the newer portion is judged.

Part 5 · Every file in the repository

What each one is and why it exists.

Application code - src/loganomaly/

File	Purpose
parse.py	Drain3 wrapper. Turns raw lines into templates.
window.py	Groups lines into windows; builds normalised count vectors.
embed.py	Pluggable embedding backends behind one interface.
detect.py	Isolation Forest, kNN distance, and the rarity control.
evaluate.py	ROC AUC, average precision, precision/recall at k.
explain.py	Prompt construction, LLM clients, response cache.
pipeline.py	Fit-once / score-many orchestration.
ingest.py	Canonicalises structured records - see Part 6.
observability.py	The structured access log. A contract with the metric filters.
api.py	FastAPI service. App factory, /health deploy gate, /score.

Delivery - .github/workflows/

File	Purpose
ci.yml	Tests, lint, shellcheck, terraform validate, image build + smoke test. Also callable by the other two.
release.yml	Tag to ghcr.io, with provenance verification.
deploy.yml	Merge to main: build, push, roll, verify, install the timer.

Deployment - deploy/

File	Purpose
deploy.sh	Rolls the container, health-gates, rolls back. Runs on the host.
test_deploy.sh	Seven rollback scenarios against stubbed docker and curl.
run_self_monitor.sh	Feeds the service's own logs back through it.
install_monitoring.sh	Installs the systemd timer. Idempotent.
self-monitor.service / .timer	The systemd units.

Why the deploy scripts live in the repo, not in user_data

The instance's first-boot script installs only the **mechanism** - Docker, directories, the environment file. The deploy **logic** is copied over fresh on every deploy.

That means changing how deployment works is a commit, not an instance rebuild. If the logic were baked into user_data, every tweak would require destroying and recreating the server.

Operations - scripts/

File	Purpose
status.sh	One command showing infrastructure, service health, alarms, detector findings and recent CI/CD runs.
self_monitor.py	The loop: read own logs, fit, score, publish, archive.
benchmark.py	Line-level detector comparison.
benchmark_windows.py	Window-size sweep.

Configuration files you should recognise

File	Purpose
pyproject.toml	Ruff and pytest configuration; the pythonpath setting is why pytest works without exporting anything.
.gitattributes	Forces LF line endings. See Part 6 - this is not cosmetic.
.dockerignore	Keeps the build context small; excludes .venv, infra, tests.
requirements.txt	Runtime dependencies.
requirements-dev.txt	Adds ruff and httpx, pinned. CI installs this one.
.terraform.lock.hcl	Pins provider versions and hashes. Committed on purpose - ignoring it is the Terraform equivalent of unpinned dependencies.

Part 6 - What broke, and what each failure teaches

The most useful part of this document. Every one of these actually happened.

Nothing below is hypothetical. Each of these was found while building this project, and most of them could only be found by running the thing against real infrastructure. When an interviewer asks what went wrong, these are the answers - and being able to describe a failure precisely is a stronger signal than describing a success.

1. The health check that could not fail

Symptom: none. Everything looked fine.

What was wrong: `/health` returned 200 OK with a body saying `"fitted": false`. The service was up, but the detector had never loaded its training data, so every real request returned 503.

Why it matters: the deployment gate polls `/health`. A container that answered 503 to every genuine request would have sailed straight through the health check and stayed live. The gate could not fail, so it was not a gate.

Fix: `/health` now returns 503 unless the pipeline can actually serve, and reports the build version and commit SHA so a deploy or rollback can be confirmed rather than assumed.

The lesson

A health check must be able to go red. If it only reports "the process is running", it tells you nothing you could not learn from ps. Ask of any check: what would have to be broken for this to fail? If the answer is "only a crash", it is not measuring the right thing.

2. The template miner deleted the only field that mattered

Symptom: the detector found nothing in its own logs. Zero anomalies, always, even during a deliberate burst of failures.

What was wrong: Drain3 masks tokens that vary between otherwise-identical lines. That is exactly right for unstructured logs. For **structured JSON** it is destructive:

```
{"ts": <*> "schema": 1, "event": "request", "method": <*>
"path": <*> "status": <*> "duration_ms": <*> "sha": <*>
```

A 200 and a 503 collapse into one template. On a 960-line sample containing a 40-request burst of 503s: **6 templates, zero detections**.

Fix: `ingest.py` canonicalises structured records into event strings whose discriminative fields are **words** - request GET /health server_error. Words survive masking; numbers do not. The similarity threshold also rose from 0.4 to 0.7, because a canonicalised event is four tokens and one differing token is a 25% difference that 0.4 absorbs.

The lesson

Understand what your preprocessing throws away. Drain3's whole purpose is discarding variation - which makes it excellent for raw syslog and actively harmful for JSON, where the varying field is the signal. A tool being well-suited to one input says nothing about the next one.

3. Isolation Forest was blind to the thing that mattered most

Symptom: after fixing the above, the burst was detected - but ranked **below two false positives**.

What was wrong: scikit-learn splits a node on a random feature, between that feature's minimum and maximum **within the training data**. A template that never appeared in the baseline is a dimension of zero variance, so no tree can ever split on it. A bad deploy's defining characteristic is log lines nobody has seen before - and the forest cannot see them.

	Score	Interpretation
The 503 burst	0.606	The actual failure
Baseline maximum	0.706	Ordinary traffic jitter

The real incident scored as LESS anomalous than normal variation.

Fix: run the library's frequency control alongside it. Inverse log-frequency scores an unseen template highly by construction, which is precisely the case the forest is blind to. A window is flagged if either signal fires, and the finding records which one did. After the fix the burst scored **6.58 against a 0.69 threshold** and ranked first, second and third.

The lesson

This is the same lesson the line-level benchmark taught: the simple frequency heuristic earns its place, and the sophisticated method does not automatically win. Knowing the failure mode of your algorithm - not just its accuracy score - is what lets you notice this.

4. The bootstrap that died before writing its configuration

Symptom: the server was running, SSH worked, Docker was installed. It looked completely healthy. But `/etc/loganomaly/env` did not exist, so every deploy would have failed on it.

What was wrong: two things compounding. A mangled line continuation left a stray `n` argument in a `curl` command, and that command was an **optional** step - installing Docker Compose for Grafana - that ran **before** the essential setup. Under `set -e`, the script exited there.

```
curl -fsSL https://...x86_64 n -o /usr/local/lib/...
curl: (6) Could not resolve host: n
```

Fix: essential setup runs first. Everything optional runs last, under `set +e`, and can only warn.

The lesson

`set -e` is right, but it makes **ordering** a correctness property. Anything non-essential must come after everything essential, or a failure in something you did not need takes down something you did. Also: a machine that responds to SSH is not a machine that works.

5. The OIDC subject claim nobody documents

Symptom: Not authorized to perform `sts:AssumeRoleWithWebIdentity`. No indication of which condition failed.

What was wrong: every AWS and GitHub example shows the subject claim as `repo:owner/name:...`. This account emits:

```
sub = repo:GibMeDaCookie09@148891370/log-anomaly-detection@1347566941:environment:production
```

GitHub's **immutable identifiers** feature embeds numeric owner and repository IDs, so that renaming a repository cannot silently transfer trust. A policy written against the documented form never matches, and the error says nothing about why.

Fix: trust both spellings, and - more importantly - the workflow now logs its own sub claim before assuming the role, so this is a two-minute diagnosis next time instead of an afternoon.

A security detail worth noticing

The tempting shortcut is a wildcard: `repo:GibMeDaCookie09*`. That would also match a repository owned by `GibMeDaCookie09x` - a username anyone can register. Both exact forms are listed instead.

6. Two requirements that contradicted each other

Symptom: every deploy failed at `Configure SSH`, five seconds in.

What was wrong: a genuine design conflict rather than a bug. "SSH only from my IP" is correct for a human operator. "GitHub runners must be able to deploy" is required for CD. Both were specified; they cannot both be true with a static firewall rule.

Fix: the deploy job opens the port for its own address, for the duration of the deploy, using short-lived OIDC credentials, and closes it in an `always()` step.

The lesson

When two requirements conflict, the weak move is to quietly relax the security one. The better move is to find the mechanism that satisfies both. Here that cost about ninety lines and made the project meaningfully stronger to talk about.

7. Line endings

Symptom: would have been bad `interpreter: /usr/bin/env bash^M`.

What was wrong: files edited on Windows picked up CRLF line endings. A shell script with CRLF fails outright on Linux, and so do the inline `run:` blocks in workflow files. Sixteen files were affected.

Fix: `.gitattributes` forcing LF, plus `shellcheck` in CI, which is what caught it.

The lesson

Cross-platform development has invisible failure modes. The tool caught in seconds what code review would not have caught at all - which is an argument for having the tool, not for reading more carefully.

8. State that lives outside your control

Symptom: `denied: permission_denied: write_package`, on a step that had worked before.

What was wrong: the repository had been deleted and recreated. The container image package **survived** that deletion, still linked to the repository that no longer existed - so the new repository had no write access to it. Creating a new package is allowed; writing to someone else's is not.

The lesson

Deleting a resource does not always delete what it created. Packages, images, DNS records and IAM roles routinely outlive the thing that made them. When you tear something down, ask what it leaves behind.

And a dozen smaller ones

What	Why it mattered
<code>.strip(" - ")</code> used for prefix removal	Strips any space or hyphen from both ends, quietly corrupting template text. <code>removeprefix</code> is what was meant.
Relative path to the training data	Starting the service from any other directory silently produced an unfitted pipeline.
Pipeline stored in a module global	Two app instances in one process tore down each other's state.
CI installed <code>requirements.txt</code> , not <code>requirements-dev.txt</code>	The API tests need <code>httpx</code> . Would have failed on the first push.
<code>ci.yml</code> concurrency group collided with itself	A push to main triggered CI directly and via <code>deploy.yml</code> ; both landed in one group with <code>cancel-in-progress</code> and killed each other.
Manual rollback always failed the SHA check	Deploying an older tag passed the current commit as expected - so the one button you need in an emergency could never succeed.
Metric namespace hardcoded in the script	Renaming the project produced an <code>AccessDenied</code> visible only on the first timer run, fifteen minutes after a clean deploy.
Terraform lock file had Windows hashes only	<code>terraform init</code> would fail for anyone on Linux or macOS.
Apostrophe in a security group rule description	AWS restricts those to a specific character set. <code>terraform validate</code> passes; only the EC2 API rejects it.
Deploy ran <code>/tmp/deploy.sh</code> after the copy step moved it	Found by reading, not running - would have failed the moment SSH worked.

Part 7 - Operating it

Standing it up, showing it off, tearing it down.

The one command that shows everything

```
bash scripts/status.sh
```

Infrastructure, service health, alarm states, SNS subscription, detector findings and recent CI/CD runs, in one view. Each section degrades independently - a missing CLI or a destroyed instance affects only its own section. A status tool that dies when something is broken is useless precisely when it is needed.

Standing it up from nothing

Full detail is in docs/SETUP.md. The short version:

Phase	What you get	Time
1. GitHub	CI green	20 min
2. Registry	Published image	10 min
3. AWS	Live infrastructure	1-2 hrs
4. Deploy	Automated deployment	20 min
5. Dashboards	Grafana, and the demo	45 min

Two things that will waste an hour if missed

Click the SNS confirmation email. Until you do, the subscription is pending and every alarm fires into a void. Terraform cannot do it for you and will show it as pending indefinitely.

DEPLOY_HOST changes on every destroy/apply cycle unless you set `allocate_elastic_ip = true`. If you are iterating on deployment, set it.

Grafana

```
scp -r monitoring ec2-user@<host>:~/
# on the host: cp .env.example .env, set a password, then
docker compose up -d

# port 3000 is NOT open in the firewall - tunnel to it
ssh -N -L 3000:localhost:3000 ec2-user@<host>
# then open http://localhost:3000
```

The tunnel is the point. An unauthenticated metrics console on the public internet is not a dashboard, it is a reconnaissance tool. `network_mode: host` in the compose file is also load-bearing: Grafana authenticates to CloudWatch through the instance metadata service, and at a hop limit of 1 a bridged container is one hop too far.

Breaking it on purpose

This is the strongest artifact the project can produce, and it is worth capturing properly. Deploy an image that cannot start - point it at training data that is not there - and watch five things happen:

1. `/health` returns **503**, not 200. The container is up; the pipeline is not fitted; the gate says so.
2. The deploy job fails and **the rollback fires**. `deploy.sh` exits 1, the summary reads "Deploy failed - rolled back", and the previous image is serving again.
3. `ServerErrors` climbs and the **api-5xx-rate alarm goes to ALARM**.
4. The next self-monitoring run **flags the burst**, with `"signals": ["rarity"]` and templates including `request GET /health server_error`.
5. `AnomaliesDetected` spikes on the dashboard and its alarm fires.

Screenshot all five. That sequence - a deployment failure detected by the thing being deployed - is the entire point of the project, and it is far more convincing shown than described.

Tearing it down

```
cd infra && terraform destroy
```

Type yes at the confirmation - nothing else is accepted. This deletes the bucket and its contents, the instance, the log group and the alarms. Re-running `app ly` rebuilds everything in about three minutes.

The GitHub side - repository, images, workflow history - costs nothing and stays. There is no reason to leave AWS running between sessions.

Part 8 · Questions you should be able to answer

If you can answer these, you understand the project.

About the design

- **Why does /health return 503 instead of 200 with a status field?** Because the deploy gate polls it. A check that cannot go red is not a check.
- **Why does the deploy verify the commit SHA as well as health?** Because a container that failed to replace its predecessor answers health perfectly - from the old version. You would report a successful deploy having shipped nothing.
- **Why do exit codes 1 and 2 differ?** One means the change did not ship. The other means nobody is serving traffic. They need different responses at 3am.
- **Why pull the image before stopping the old container?** So a registry outage fails the deploy without causing an outage.
- **Why is the deploy logic in the repo rather than in user_data?** So changing it is a commit rather than an instance rebuild.

About the infrastructure

- **What does least privilege mean here, specifically?** No managed policies, no resource wildcards, no CreateLogGroup, and ListBucket restricted by prefix condition because it cannot be restricted by object ARN.
- **Why IMDSv2 required rather than optional?** Because with v1 still accepted, any SSRF in the application can read the metadata endpoint and steal the instance's credentials - the Capital One mechanism.
- **Why the default VPC and not a private subnet?** A private subnet needs a NAT gateway to reach the registry, and that is about \$32 a month - more than everything else combined.
- **Why is the Terraform lock file committed?** It pins provider versions and hashes so `init` resolves identically everywhere. Ignoring it is the equivalent of shipping unpinned dependencies.
- **How does the pipeline authenticate to AWS?** OIDC. A short-lived signed token, verified against a trust policy pinning the audience and repository. No stored credential.

About the detection

- **Where does the accuracy come from?** Window aggregation. F1 goes from 0.133 at line level to 0.765 at windows of 50 - a 5.8x improvement. Everything else is a rounding error.
- **Why not let the language model do the detection?** It would be non-reproducible, unauditable, and expensive per log line. Detection is statistical; the model only explains what was already flagged.
- **Why is there a detector that uses no embeddings at all?** As a control. It makes "the sophisticated approach helped" testable rather than assumed - and at line level it was not true.
- **Are the labels used for training?** No. Evaluation only. In production there are no labels; training on them would make it a classifier, which is a different and much easier problem.

The question to have a real answer for

What went wrong, and what did you do about it?

Pick two from Part 6. The Isolation Forest one is the strongest technical answer - it shows you understand an algorithm's failure mode and not just its accuracy. The SSH-versus-runners one is the strongest engineering answer - it shows you resolved conflicting requirements instead of quietly weakening the security one.

Part 9 · Glossary

Term	Meaning
AMI	Amazon Machine Image. The operating system template an EC2 instance boots from.
ARN	Amazon Resource Name. The globally unique ID of an AWS resource.
CIDR	Notation for an IP range. /32 is one address; 0.0.0.0/0 is everything.
Container	A running instance of an image.
Drain3	The log parsing library that groups similar lines into templates.
EC2	Amazon's virtual machines.
Elastic IP	A fixed public address. Free while attached to a running instance; billed when not.
F1 score	The harmonic mean of precision and recall. One number summarising accuracy on an imbalanced problem.
ghcr.io	GitHub Container Registry.
HCL	HashiCorp Configuration Language. What Terraform files are written in.
IAM	AWS Identity and Access Management. Who may do what.
Idempotent	Safe to run repeatedly. Running it twice leaves the same result as once.
IMDS	Instance Metadata Service. The endpoint at 169.254.169.254 that hands an instance its own credentials.
Image	A read-only package containing an application and its environment.
Isolation Forest	An outlier detector. Points isolated by few random splits are unusual.
Least privilege	Granting exactly the permissions needed and no more.
OIDC	OpenID Connect. Proving identity with a signed token instead of a stored password.
Registry	A warehouse for container images.
Rollback	Restoring the previous version after a failed deploy.
Runner	The throwaway virtual machine a CI job runs on.
S3	Amazon's object storage. Buckets of files addressed by key.
Security group	A firewall attached to an AWS resource. Default-deny.
shellcheck	A static analyser for shell scripts.
SNS	Simple Notification Service. Publish/subscribe messaging.
SSRF	Server-Side Request Forgery. Tricking a server into fetching a URL an attacker chose.
systemd timer	The modern Linux replacement for cron.
Template	A log line with its variable parts replaced by wildcards.
Terraform	The tool that turns infrastructure descriptions into real resources.

Term	Meaning
Window	A consecutive run of log lines scored as one unit.